

AI-Powered Face Recognition Surveillance System - Dashboard

COMPONEN DESIGN

5.2 Video Ingestion Module

Processing with Multiple Cameras
Capture and Ingest Video Streams

- Use FFmpeg, GStreamer, or OpenCV to capture video from multiple cameras.
- Stream video via RTSP.
- Assign a unique identifier to each camera.
- Since multiple cameras are involved, use threading or multiprocessing for parallel processing

```
import cv2
import threading

camera_sources = [
    "rtsp://username:password@camera1_ip:554/stream",
    "rtsp://username:password@camera2_ip:554/stream"
]

def process_camera(rtsp_url, cam_id):
    cap = cv2.VideoCapture(rtsp_url)
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow(f"Camera {cam_id}", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    cap.release()
    cv2.destroyAllWindows()

# Run each camera in a separate thread
threads = []
for i, cam_url in enumerate(camera_sources):
```

```

t = threading.Thread(target=process_camera, args=(cam_url, i))
threads.append(t)
t.start()

for t in threads:
    t.join()

```

Streaming RTSP to Web Dashboard

- Web browsers don't support RTSP directly, so you need to convert RTSP to WebRTC or HLS. Using FFmpeg to Convert RTSP → WebRTC

```

ffmpeg -i rtsp://username:password@camera_ip:554/stream -c:v libx264 -f
webm - | nc -l -p 9000

```

- This allows a browser to fetch video from port 9000 via WebRTC.
- Can be integrated with WebSockets for real-time updates.

Streaming RTSP to Web Dashboard (Op)

5.3 AI Processing Module

Processing 3,000 CCTV feeds, you need AI servers to handle:

- Real-time object detection (People, vehicles, faces, intrusions)
- Face recognition (Attendance, security, blacklist detection)
- Anomaly detection (Unusual behavior, unattended objects)
- Multi-stream AI processing (Handling hundreds of streams efficiently)

Key Factor :

- High GPU Performance – AI workloads require powerful GPUs for deep learning.
- Low Latency Processing – AI must process CCTV feeds in real-time.
- Scalability – The system must handle thousands of cameras efficiently.

AI Server Options for CCTV

AI Server	Best For	Pros	Cons
NVIDIA DGX A100	Large-scale AI processing	✓ 5+ AI GPUs ✓ 10,000+ streams	✗ Expensive (\$100K+)

NVIDIA Tesla A100	Deep learning AI tasks	✓ High-speed AI ✓ Scalable clusters	✗ Requires advanced setup
Jetson AGX Orin	Edge AI for smart cameras	✓ Low power ✓ On-camera AI	✗ Limited processing power
Intel Xeon + VPU	CPU-based AI inference	✓ Good for small setups ✓ Works with OpenVINO	✗ Slower than GPU
TPU (Google Coral)	Small-scale AI	✓ Low cost ✓ Edge-based	✗ Not good for 3,000 cameras

5.3.1 Multi-stream AI processing (Handling hundreds of streams efficiently)

GPU-Accelerated AI Models

- TensorRT + NVIDIA A100 → For high-speed face & object detection
- YOLOv8 or OpenCV DNN → Lightweight AI models for real-time processing
- DeepStream SDK → Multi-stream AI inference for 100+ cameras

AI Pipelines for WebRTC Surveillance

- RTSP Input → CCTV streams from cameras
- FFmpeg/GStreamer → Converts RTSP to AI-compatible format
- AI Model Inference (GPU) → Object & face detection
- Database (PostgreSQL + pgvector) → Stores embeddings & results
- WebRTC (Mediasoup) → Streams processed video to dashboard
- WebSockets → Sends real-time alerts

AI Server Setup

Component	Recommended Setup
AI Server	NVIDIA DGX A100 + TensorRT + DeepStream
AI Processing	YOLOv8 + FaceNet + OpenCV DNN
Database	PostgreSQL + pgvector for face embeddings
WebRTC Streaming	Mediasoup + WebSockets for real-time AI alerts

Process Frames in Parallel (AI Inference)

- Use Threading or Multiprocessing to handle multiple streams.

- Use GPU Acceleration (TensorRT, OpenVINO, CUDA) for real-time inference.

```
import threading

def process_frame(camera_id, frame):
    # Run AI model (Face Recognition, Object Detection, etc.)
    detected_objects = ai_model.detect(frame)
    print(f"Camera {camera_id} detected: {detected_objects}")

threads = []
for i, frame in enumerate(frames):
    t = threading.Thread(target=process_frame, args=(i, frame))
    threads.append(t)
    t.start()

# Wait for all threads to complete
for t in threads:
    t.join()
```

5.3.2 Basic Motion Detection

Basic Motion Detection

```
import cv2

fgbg = cv2.createBackgroundSubtractorMOG2()

def process_frame(frame):
    fgmask = fgbg.apply(frame)
    contours, _ = cv2.findContours(fgmask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    for contour in contours:
        if cv2.contourArea(contour) > 500:
            x, y, w, h = cv2.boundingRect(contour)
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
    return frame
```

5.3.3 Real-time object detection (People, vehicles, faces, intrusions)

System Architecture :

 RTSP Cameras →  Video Ingestion (YOLOv8 on GPU) →  FastAPI Backend (WebSockets for real-time alerts) →  Web Dashboard (Svelte/WebRTC/WebSockets)

Install Dependencies

```
pip install ultralytics fastapi uvicorn opencv-python numpy websockets aiohttp
```

Object Detection with YOLOv8 (Using GPU)

Create a Python script to process RTSP streams and detect people, vehicles, and faces in real-time.

```
import cv2
import torch
import asyncio
import websockets
from ultralytics import YOLO

# Load YOLOv8 model (COCO dataset includes people, vehicles)
model = YOLO("yolov8n.pt") # Use 'yolov8s.pt' for better accuracy

# RTSP Camera Source
RTSP_URL = "rtsp://username:password@camera_ip:port/stream"

# WebSocket URL for sending detections
WEBSOCKET_URL = "ws://localhost:8000/ws"

async def process_stream():
    cap = cv2.VideoCapture(RTSP_URL)

    if not cap.isOpened():
        print("Error: Cannot open RTSP stream")
        return

    async with websockets.connect(WEBSOCKET_URL) as ws:
        while True:
            ret, frame = cap.read()
            if not ret:
                break

            # Run YOLO detection
            results = model(frame)

            detections = []
```

```

        for result in results.xyxy[0]: # Bounding boxes (x1, y1, x2,
y2, confidence, class)
            x1, y1, x2, y2, conf, cls = result.tolist()
            class_name = model.names[int(cls)]

            # Filter specific objects (people, vehicles)
            if class_name in ["person", "car", "truck", "motorbike"]:
                detections.append({
                    "object": class_name,
                    "confidence": round(conf, 2),
                    "bbox": [int(x1), int(y1), int(x2), int(y2)]
                })

            # Send detections via WebSocket
            if detections:
                await ws.send(str(detections))

            # Display detections locally
            for det in detections:
                x1, y1, x2, y2 = det["bbox"]
                label = f"{det['object']} ({det['confidence']})"
                cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
                cv2.putText(frame, label, (x1, y1 - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

            cv2.imshow("AI CCTV Detection", frame)
            if cv2.waitKey(1) & 0xFF == ord("q"):
                break

        cap.release()
        cv2.destroyAllWindows()

# Run the detection loop
asyncio.run(process_stream())

```

5.3.4 Face recognition (Attendance, security, blacklist detection)

Face Recognition Pipeline

1. RTSP Camera → AI Server (YOLO + FaceNet)
2. AI detects faces → Matches with database (PostgreSQL + pgvector)
3. WebSocket sends real-time alerts
4. WebRTC streams processed video to dashboard

Running YOLOv8 + FaceNet on GPUs

Run YOLOv8 on Multiple RTSP Feeds

```
from ultralytics import YOLO
import cv2

model = YOLO("yolov8n.pt")

# Process multiple RTSP streams
rtsp_urls = [
    "rtsp://camera1_ip/live",
    "rtsp://camera2_ip/live",
    "rtsp://camera3_ip/live"
]

for rtsp_url in rtsp_urls:
    cap = cv2.VideoCapture(rtsp_url)
    ret, frame = cap.read()
    if ret:
        results = model(frame)
        print(results)
```

Optimize Face Recognition with TensorRT

TensorRT speeds up inference by 5x-10x on GPUs

```
import tensorrt as trt

TRT_LOGGER = trt.Logger()
with trt.Builder(TRT_LOGGER) as builder:
    # Load optimized FaceNet model
    engine = builder.build_cuda_engine()
```

Face Recognition Implementation (Attendance, Security, Blacklist Detection)

This solution will use MediaPipe for face detection, FaceNet for face embeddings, and pgvector (PostgreSQL) for storage and matching. The backend will be built with FastAPI and use WebSockets for real-time alerts.

System Architecture

 RTSP Cameras →  Face Detection (MediaPipe/YOLO) →  Face Recognition

(FaceNet) →  Database (PostgreSQL + pgvector for embeddings) →  FastAPI
API (WebSockets for alerts) →  Web Dashboard (Svelte)

Install Dependencies

```
pip install opencv-python numpy fastapi uvicorn mediapipe torch torchvision  
pgvector psycpg2 websockets
```

Face Detection & Embeddings with MediaPipe + FaceNet

This script extracts face embeddings from CCTV footage and checks for attendance or blacklist detection.

```
import cv2  
import numpy as np  
import torch  
import mediapipe as mp  
import psycpg2  
import asyncio  
import websockets  
from torchvision import transforms  
from facenet_pytorch import InceptionResnetV1  
  
# Load MediaPipe Face Detector  
mp_face_detection = mp.solutions.face_detection  
face_detector =  
mp_face_detection.FaceDetection(min_detection_confidence=0.6)  
  
# Load FaceNet model for face recognition  
face_recognition_model = InceptionResnetV1(pretrained='vggface2').eval()  
  
# PostgreSQL connection (pgvector for embedding storage)  
conn = psycpg2.connect("dbname=facerecognition user=postgres  
password=yourpassword host=localhost")  
cursor = conn.cursor()  
  
# RTSP Camera Source  
RTSP_URL = "rtsp://username:password@camera_ip:port/stream"  
  
async def process_stream():  
    cap = cv2.VideoCapture(RTSP_URL)  
  
    if not cap.isOpened():  
        print("Error: Cannot open RTSP stream")
```



```

    return

    async with websockets.connect("ws://localhost:8000/ws") as ws:
        while True:
            ret, frame = cap.read()
            if not ret:
                break

            # Convert to RGB
            rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

            # Detect faces
            results = face_detector.process(rgb_frame)

            if results.detections:
                for detection in results.detections:
                    bbox = detection.location_data.relative_bounding_box
                    x, y, w, h = (int(bbox.xmin * frame.shape[1]),
                                int(bbox.ymin * frame.shape[0]),
                                int(bbox.width * frame.shape[1]),
                                int(bbox.height * frame.shape[0]))

                    face_crop = rgb_frame[y:y+h, x:x+w]
                    if face_crop.size == 0:
                        continue

                    # Preprocess face for FaceNet
                    face_tensor =
transforms.ToTensor()(cv2.resize(face_crop, (160, 160)))
                    face_tensor = face_tensor.unsqueeze(0)

                    # Extract face embeddings
                    with torch.no_grad():
                        embedding =
face_recognition_model(face_tensor).numpy().tolist()

                    # Compare with database (attendance or blacklist)
                    cursor.execute("SELECT id, name FROM faces WHERE
embedding <=> %s ORDER BY embedding <=> %s LIMIT 1;",
                                (embedding, embedding))
                    result = cursor.fetchone()

                    if result:

```

```

        person_id, person_name = result
        status = "ATTENDANCE" if person_id in
registered_users else "BLACKLISTED"

        # Send event to WebSocket
        await ws.send(f'{{"name": "{person_name}",
"status": "{status}"}}')

        # Draw on frame
        color = (0, 255, 0) if status == "ATTENDANCE" else
(0, 0, 255)

        cv2.rectangle(frame, (x, y), (x+w, y+h), color, 2)
        cv2.putText(frame, f"{person_name} ({status})", (x,
y - 10),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.5, color,
2)

        cv2.imshow("Face Recognition", frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break

    cap.release()
    cv2.destroyAllWindows()

asyncio.run(process_stream())

```

5.4 Event Storage

Event Handling and Storage

- Store AI detection results in a PostgreSQL (pgvector).
- Store video clips in local/NAS storage or cloud storage (S3, Google Drive).

```

import psycopg2

conn = psycopg2.connect("dbname=surveillance user=admin password=secret")
cur = conn.cursor()

def save_event(camera_id, detected_objects):
    query = "INSERT INTO detections (camera_id, objects, timestamp) VALUES
(%, %, NOW())"

```

```
cur.execute(query, (camera_id, detected_objects))
conn.commit()
```

5.5 Notification System

Real-time Notifications & Alerts

- Use WebSockets for real-time updates on the dashboard.
- Send alerts via email, SMS, or mobile push notifications when a critical event is detected.

```
import smtplib
```

```
def send_alert(camera_id, detected_object):
    message = f"Alert: {detected_object} detected on Camera {camera_id}"
    server = smtplib.SMTP("smtp.mailserver.com", 587)
    server.login("your_email", "password")
    server.sendmail("your_email", "user_email", message)
    server.quit()
```

Implement WebSockets for event-based updates

Use WebSockets for event-based updates (motion detection alerts, face recognition results).

WebSocket Server (FastAPI)

This will send real-time alerts when AI detects something.

```
from fastapi import FastAPI, WebSocket
import json
```

```
app = FastAPI()
clients = []
```

```
@app.websocket("/ws")
```

```
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    clients.append(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            for client in clients:
                await client.send_text(json.dumps({"event": "motion
detected"}))
```

```
except:
    clients.remove(websocket)
```

5.6 Streaming & Storage Server

The Streaming and Storage Server plays a crucial role in handling 3,000 CCTV cameras by efficiently storing, retrieving, and streaming AI-processed video feeds. This section breaks down the best technologies, storage architectures, and streaming strategies to optimize performance.

Streaming Server Design
Handles real-time video delivery from AI-processed CCTV feeds

Streaming Protocol Choice
For low-latency, high-quality video streaming, we use WebRTC + RTSP:

Protocol	Use Case	Why Chosen?
RTSP (Real-Time Streaming Protocol)	Camera to AI Processing	Standard protocol for CCTV feeds
WebRTC (Mediasoup SFU)	AI-Processed Video to Clients	Low-latency, real-time playback
HLS (HTTP Live Streaming)	On-Demand Playback	Suitable for recorded video but adds latency

WebRTC-Based Streaming Architecture :

- Ingest RTSP video from cameras → AI Processing → WebRTC for client playback

Streaming Path:
Low-latency streaming for AI surveillance dashboard.
 Cameras (RTSP) →  Video Ingestion Module (FFmpeg/GStreamer/NVIDIA DeepStream for AI Processing) →  Streaming Server (WebRTC or HLS) →  Dashboard (WebRTC Player)

Storage Server Design
Handles video recording, AI metadata, and event storage

Storage Requirements
To store and retrieve high-resolution AI-processed videos, we need:
Hybrid storage approach: SSD cache for real-time, HDD for archival.

Requirement	Recommended Solution	Reason
-------------	----------------------	--------

High-Speed Video Writes	NVMe SSDs (Cache Layer)	Handles high-speed writes from 3,000 cameras
Long-Term Storage	HDD RAID Arrays (NAS/SAN)	Cost-efficient for bulk video storage
AI Metadata Storage	PostgreSQL + pgvector	Efficient for AI event lookups
Event Indexing	Elasticsearch + Kibana	Fast search & visualization

Storage Architecture
Video Storage Path:
Cameras → AI Server → NVMe SSD Cache → HDD Archive (NAS/SAN)
RAID 6 or RAID 10 recommended for redundancy & speed.

Storage Type	Use Case	Recommended Technology
NVMe SSD (Cache Layer)	Temporary storage for active video feeds	Samsung PM9A3 NVMe (U.2)
RAID HDD (Main Storage)	Long-term video storage	Seagate Exos 18TB HDD (RAID 6)
Object Storage (Cloud/On-Prem)	Backup & archival storage	MinIO / AWS S3

- Video Retention Policy
- SSD (Cache Layer) → Stores last 24-48 hours of video for quick retrieval.
 - HDD (NAS/SAN Storage) → Stores last 30-90 days of recordings.
 - Cloud Backup (Optional) → Stores archived events for 1+ years.
 - AI-based Smart Storage: Only store footage when AI detects an event (motion, face match, etc.).

- Network Considerations for Streaming & Storage
- Storage Network: 40GbE or 100GbE Fiber between storage servers for high-speed data transfer.
 - Streaming Network: 10GbE Backbone for WebRTC & RTSP streaming.
 - AI Processing: Separate 10GbE AI network to prevent congestion.

- Best Practice :
- Use WebRTC (Mediasoup) for real-time AI video streaming.
 - Deploy a hybrid storage solution (SSD for cache, HDD for archive).
 - Use PostgreSQL for AI metadata, Elasticsearch for event search.
 - Ensure high-speed 40GbE+ networking for video storage & retrieval.
 - Optimize storage with AI-triggered event-based recording.

5.7 Backend API

FastAPI Backend with WebSockets

This API receives and broadcasts detections in real-time to connected clients (dashboard or mobile apps).

```
from fastapi import FastAPI, WebSocket
from fastapi.middleware.cors import CORSMiddleware
import json

app = FastAPI()

# Allow frontend to access API
app.add_middleware(CORSMiddleware, allow_origins=["*"],
allow_credentials=True, allow_methods=["*"], allow_headers=["*"])

# Store active WebSocket clients
active_connections = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    active_connections.append(websocket)

    try:
        while True:
            data = await websocket.receive_text()
            detection_data = json.loads(data)

            # Broadcast detections to all clients
            for conn in active_connections:
                await conn.send_text(json.dumps(detection_data))
    except:
        active_connections.remove(websocket)

# Run with: uvicorn main:app --host 0.0.0.0 --port 8000
```

FastAPI Backend for WebSockets & Face Registration

This backend handles face registration, real-time alerts, and WebSocket streaming.

```
from fastapi import FastAPI, WebSocket
from fastapi.middleware.cors import CORSMiddleware
import json
import psycpg2
```

```

app = FastAPI()

# PostgreSQL connection
conn = psycopg2.connect("dbname=facerecognition user=postgres
password=yourpassword host=localhost")
cursor = conn.cursor()

# Allow frontend to access API
app.add_middleware(CORSMiddleware, allow_origins=["*"],
allow_credentials=True, allow_methods=["*"], allow_headers=["*"])

# Store active WebSocket clients
active_connections = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    active_connections.append(websocket)

    try:
        while True:
            data = await websocket.receive_text()
            for conn in active_connections:
                await conn.send_text(data)
    except:
        active_connections.remove(websocket)

@app.post("/register_face")
async def register_face(name: str, embedding: list):
    cursor.execute("INSERT INTO faces (name, embedding) VALUES (%s, %s);",
(name, embedding))
    conn.commit()
    return {"message": "Face registered"}

# Run with: uvicorn main:app --host 0.0.0.0 --port 8000

```

5.8 Dashboard

The Dashboard is the control center for monitoring, analyzing, and managing 3,000 CCTV cameras with AI-powered insights. It provides real-time video feeds, AI alerts,

analytics, and system controls through a fast, responsive web-based interface.

Dashboard Page List

- 1 Login & Authentication
 - ✓ Login Page (Username, Password, 2FA)
 - ✓ Forgot Password Page
- 2 Home & Live Monitoring
 - ✓  Dashboard Overview (System Summary, Active Cameras, Alerts, Statistics)
 - ✓  Live Camera Streams (Multi-Camera Grid View)
 - ✓  Full-Screen & Camera Switching
- 3 AI-Powered Alerts & Notifications
 - ✓  Alerts Page (Motion, Blacklist, Intrusions, Unattended Objects)
 - ✓  Notification Center (Live Notifications with Sound & Popup)
- 4 Face Recognition & Attendance
 - ✓  Face Recognition Log (Identified Faces, Confidence Score)
 - ✓  Attendance Report (Check-In/Out, Timestamps, Status)
 - ✓  Blacklist Person Detection Log
- 5 Video Playback & Storage
 - ✓  Video Archive (Recorded Events, Date/Time Filtering)
 - ✓  Image Snapshot Viewer
 - ✓  Search & Filter (Person, Vehicle, Camera, Date)
- 6 Reports & Analytics
 - ✓  Security Incident Reports
 - ✓  Heatmaps (Motion, Crowd Density, Unusual Behavior)
 - ✓  Export Reports (CSV, PDF, Excel)
- 7 Camera & System Management
 - ✓  Camera List & Status (RTSP Stream, Health Monitoring)
 - ✓  System Settings (Storage, AI Sensitivity, Alert Thresholds)
 - ✓  Notification Preferences
- 8 User & Role Management
 - ✓  User List (Admin, Security Staff, Operators)
 - ✓  Blacklist Management (Add/Remove Blacklisted Faces)
 - ✓  Roles & Permissions (Access Control)

Key Features of the Dashboard :

Modular & Extensible → Can add more AI analytics (e.g., vehicle recognition, crowd analysis).

Feature	Purpose	Why It's Important?
Live CCTV	Real-time WebRTC video feeds from AI-processed	Enables security teams to

Streaming	cameras	monitor events live
AI Alerts & Notifications	Instant alerts for face recognition, object detection, intrusion detection, etc.	Immediate response to security threats
Event Logs & Search	Stores past incidents with timestamps, images, and metadata	Helps in forensic analysis
Facial Recognition Search	Search for faces detected across cameras	Critical for law enforcement & security teams
Multi-User Access	Role-based access control (RBAC) for admins, operators, and analysts	Ensures secure usage
System Health Monitoring	Monitor AI server load, GPU usage, camera status, and network health	Prevents downtime
Reporting & Analytics	Generate reports on activity trends, security incidents, and system performance	Helps in decision-making

Tech Stack for the Dashboard

Component	Technology	Why Chosen?
Frontend	SvelteKit + TailwindCSS	Fast, lightweight, and optimized for real-time UI
Backend	FastAPI (Python)	High-performance API handling for AI events & video streams
Database	PostgreSQL + pgvector	Stores AI detections, face embeddings, and metadata efficiently
WebSockets	FastAPI + Redis Pub/Sub	Enables real-time AI alerts and updates
Streaming	WebRTC (Mediasoup)	Ensures low-latency CCTV video streaming
AI Search	Elasticsearch	Enables fast search across recorded events
Authentication	OAuth2 / JWT	Ensures secure multi-user access

UI/UX Design Considerations

- Live CCTV Grid View
Displays real-time WebRTC feeds from multiple cameras.

- Supports dynamic layouts (1, 4, 9, or 16 cameras per screen).
- Click on a feed to view full-screen with AI annotations.
- Optimized for security teams monitoring multiple cameras simultaneously.
- AI Alerts & Event Timeline
 - Live alerts for face matches, motion detection, or suspicious objects.
 - Alert timestamp, image snapshot, and AI confidence score.
 - Filters: Time range, location, event type (e.g., intrusion, unknown person, etc.).
 - Real-time response ensures security personnel can act immediately.
- Facial Recognition Search
 - Search for a person's face detected in any camera.
 - Enter image or use past detections for searching.
 - Returns list of occurrences, timestamps, and camera locations.
 - Useful for investigations & tracking security threats.
- System Health Dashboard
 - Displays AI server performance (GPU load, storage usage, processing speed, etc.).
 - Shows camera status (online/offline, stream quality, bitrate).
 - Monitors WebRTC & RTSP streaming performance.
 - Prevents system failures with proactive monitoring.

Scalability & Deployment
 Supports thousands of concurrent users with high performance.

Aspect	Solution
Web App Deployment	Dockerized SvelteKit & FastAPI running on Kubernetes
Load Balancing	Nginx + Cloudflare for high availability
Real-Time Processing	WebSockets & Kafka for AI event streaming
Database Scaling	PostgreSQL + Redis for fast queries

- Best Practices for Dashboard
- Use WebRTC for real-time CCTV streaming.
 - Deploy a fast frontend (SvelteKit) for real-time responsiveness.
 - Store AI event data efficiently using PostgreSQL + pgvector.
 - Use WebSockets for real-time alerts & status updates.
 - Monitor system health to prevent failures.

Web Dashboard (WebSockets + Svelte)
 This Svelte frontend will receive and display real-time alerts.

HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

UI mockup plan for the AI-Powered CCTV Surveillance Dashboard. It includes key sections like Live Monitoring, AI Alerts, Facial Recognition Search, and System Health Monitoring.



6.2 Screen Images

Functionality :

- Live Camera Grid with AI-detected annotations
- AI Alerts Panel for real-time notifications
- Facial Recognition Search for identifying individuals
- System Health Monitor tracking GPU, storage, and uptime
- Event Timeline for reviewing past incidents

6.3 Screen Objects and Actions

1. Live Camera Feeds Section



Objects:

- Camera Tiles – Displays live CCTV streams (grid layout).
- AI Detection Overlay – Bounding boxes for detected people, vehicles, intrusions, and unattended objects.
- Camera Selection Dropdown – Switch between different CCTV feeds.

♦ Actions:

- Click on a camera tile → Expand to full screen.
- Hover over AI overlay → Show detection details (e.g., person ID, vehicle type).
- Click on camera selection → Switch to another feed.

2. AI Event & Alert Panel



Objects:

- Real-time Alerts List – Log of AI-detected events (e.g., "Intrusion detected at Gate 3").
- Event Filters – Dropdowns for filtering by event type (motion, face recognition, anomaly, etc.).
- Acknowledge Button – Mark an event as reviewed.

♦ Actions:

- Click on an alert item → Open event details with timestamped video clip.
- Use filter dropdowns → Show only selected event types.
- Click acknowledge → Dismiss the alert and mark it as resolved.

3. Face Recognition Section



Objects:

- Face Match Panel – Shows detected face with match confidence.
- Blacklist & Attendance Status – Indicates if a person is on a watchlist or marked present.
- Search Bar – Look up individuals by name or ID.

♦ Actions:

- Click on a face match result → View detailed profile (past detections, logs).
- Click add to blacklist → Mark a detected person as a security risk.
- Use search bar → Retrieve past face recognition records.

4. Incident Playback & Video Storage



Objects:

- Playback Timeline – Interactive timeline for reviewing recorded footage.
- Event Markers – Highlights AI-detected incidents on the timeline.
- Download Button – Export video clips.
- ♦ Actions:
 - Click on an event marker → Jump to the specific incident in playback.
 - Adjust timeline slider → Navigate through video history.
 - Click download → Save a video segment for offline analysis.

5. System Settings & Admin Controls



Objects:

- User Management – Manage security personnel accounts and roles.
- AI Model Settings – Adjust sensitivity for object detection, face recognition.
- Storage & Backup Configuration – Set cloud/local video storage preferences.

Actions:

- Click add user → Create a new security/admin account.
- Adjust AI settings → Fine-tune detection thresholds.
- Set storage limits → Define video retention policies.

DATA DESIGN

4.1 Data Description

Entity-Relationship Diagram (ERD) for the AI-Powered Face Recognition Surveillance System.

- Users – Stores registered individuals (employees, VIPs, or blacklisted persons).
- Faces – Stores face embeddings for recognition.
- Cameras – Stores CCTV camera sources (RTSP URLs, locations).
- Attendance – Logs recognized individuals for attendance tracking.
- Alerts – Stores blacklist detections, unusual behavior, and security alerts.
- Motion Events – Detects and logs motion events (people, vehicles, objects).
- Unattended Objects – Detects and logs objects left unattended.
- Vehicle Logs – Recognizes and tracks vehicles in the surveillance area.
- Video/Image Storage – Saves video clips and snapshots of incidents.
- Notifications – Sends real-time alerts to security personnel.

Users	Faces	Cameras	Attendance	Alerts
user_id (PK)	face_id (PK)	camera_id (PK)	attendance_id(PK)	alert_id (PK)
name	user_id (FK)	location	user_id (FK)	user_id (FK)
role	embedding	rtsp_url	camera_id (FK)	camera_id (FK)
status	registered_at	active	timestamp	alert_type
created_at			status	timestamp

+-----+			+-----+	+-----+
+-----+	+-----+	+-----+	+-----+	+-----+
MotionEvents	UnattendedObj	Vehicles	VideoStorage	Notifications
+-----+	+-----+	+-----+	+-----+	+-----+
event_id (PK)	obj_id (PK)	vehicle_id(PK)	video_id (PK)	notif_id (PK)
camera_id (FK)	camera_id (FK)	camera_id (FK)	camera_id (FK)	user_id (FK)
timestamp	timestamp	timestamp	video_path	alert_id (FK)
event_type	obj_type	plate_number	image_path	message
image_path	image_path	vehicle_type	detected_at	sent_at
+-----+	+-----+	+-----+	+-----+	+-----+

PostgreSQL Table Definitions

```
-- Enable vector extension for face recognition
```

```
CREATE EXTENSION IF NOT EXISTS vector;
```

```
-- Users Table (Tracks registered individuals)
```

```
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    role TEXT CHECK (role IN ('Employee', 'VIP', 'Blacklisted')),
    status TEXT CHECK (status IN ('Active', 'Inactive')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Faces Table (Stores embeddings for recognition)
```

```
CREATE TABLE faces (
    face_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(user_id) ON DELETE CASCADE,
    embedding vector(512), -- 512-dimensional face embedding
    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Cameras Table (Stores CCTV RTSP sources)
```

```
CREATE TABLE cameras (
    camera_id SERIAL PRIMARY KEY,
    location TEXT NOT NULL,
    rtsp_url TEXT UNIQUE NOT NULL,
    active BOOLEAN DEFAULT TRUE
);
```

```
-- Attendance Table (Logs recognized attendance)
```



```

CREATE TABLE attendance (
    attendance_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(user_id) ON DELETE CASCADE,
    camera_id INTEGER REFERENCES cameras(camera_id) ON DELETE CASCADE,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status TEXT CHECK (status IN ('Present', 'Late', 'Absent'))
);

-- Alerts Table (Logs blacklist detections, intrusions, and security
events)
CREATE TABLE alerts (
    alert_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(user_id) ON DELETE CASCADE,
    camera_id INTEGER REFERENCES cameras(camera_id) ON DELETE CASCADE,
    alert_type TEXT CHECK (alert_type IN ('Blacklist Detected',
'Unauthorized Entry', 'Unusual Behavior')),
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Motion Detection Table (Logs motion events for objects, people,
vehicles)
CREATE TABLE motion_events (
    event_id SERIAL PRIMARY KEY,
    camera_id INTEGER REFERENCES cameras(camera_id) ON DELETE CASCADE,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    event_type TEXT CHECK (event_type IN ('Person', 'Vehicle', 'Object')),
    image_path TEXT
);

-- Unattended Object Table (Logs unattended objects detection)
CREATE TABLE unattended_objects (
    obj_id SERIAL PRIMARY KEY,
    camera_id INTEGER REFERENCES cameras(camera_id) ON DELETE CASCADE,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    obj_type TEXT CHECK (obj_type IN ('Bag', 'Box', 'Other')),
    image_path TEXT
);

-- Vehicle Detection Table (Logs vehicle recognition and license plate
detection)
CREATE TABLE vehicles (
    vehicle_id SERIAL PRIMARY KEY,
    camera_id INTEGER REFERENCES cameras(camera_id) ON DELETE CASCADE,

```

```

        timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        plate_number TEXT UNIQUE,
        vehicle_type TEXT CHECK (vehicle_type IN ('Car', 'Truck',
'Motorcycle')),
        image_path TEXT
    );

-- Video and Image Storage Table (Stores evidence of security events)
CREATE TABLE video_storage (
    video_id SERIAL PRIMARY KEY,
    camera_id INTEGER REFERENCES cameras(camera_id) ON DELETE CASCADE,
    detected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    video_path TEXT,
    image_path TEXT
);

-- Notifications Table (Sends real-time security alerts)
CREATE TABLE notifications (
    notif_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(user_id) ON DELETE CASCADE,
    alert_id INTEGER REFERENCES alerts(alert_id) ON DELETE CASCADE,
    message TEXT,
    sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

4.2 Data Dictionary

Data Dictionary for AI-Powered CCTV Surveillance System. It defines each table, its attributes, data types, and descriptions.

Table Name	Column Name	Data Type	Description
users	user_id	SERIAL PRIMARY KEY	Unique identifier for each user
	name	TEXT	Full name of the user
	role	TEXT	Role of the user (Employee, VIP, Blacklisted)
	status	TEXT	Active or Inactive status

	created_at	TIMESTAMP	User registration date
faces	face_id	SERIAL PRIMARY KEY	Unique identifier for each stored face
	user_id	INTEGER FOREIGN KEY	Links face to a registered user
	embedding	VECTOR(512)	Face embedding for recognition
	registered_at	TIMESTAMP	Time when the face was registered
cameras	camera_id	SERIAL PRIMARY KEY	Unique ID for each camera
	location	TEXT	Camera location description
	rtsp_url	TEXT UNIQUE	RTSP URL for video streaming
	active	BOOLEAN	Whether the camera is active
attendance	attendance_id	SERIAL PRIMARY KEY	Unique attendance record ID
	user_id	INTEGER FOREIGN KEY	Recognized user's ID
	camera_id	INTEGER FOREIGN KEY	Camera where user was detected
	timestamp	TIMESTAMP	Time of detection
	status	TEXT	Attendance status (Present, Late, Absent)
alerts	alert_id	SERIAL PRIMARY KEY	Unique ID for each security alert
	user_id	INTEGER FOREIGN KEY	Detected user (if applicable)
	camera_id	INTEGER FOREIGN KEY	Camera where alert was triggered
	alert_type	TEXT	Type of alert (Blacklist, Intrusion, Unusual Behavior)
	timestamp	TIMESTAMP	Time of alert

motion_events	event_id	SERIAL PRIMARY KEY	Unique ID for each motion detection event
	camera_id	INTEGER FOREIGN KEY	Camera detecting motion
	timestamp	TIMESTAMP	Time of detection
	event_type	TEXT	Type of motion (Person, Vehicle, Object)
	image_path	TEXT	Snapshot of detected motion
unattended_objects	obj_id	SERIAL PRIMARY KEY	Unique ID for unattended object detection
	camera_id	INTEGER FOREIGN KEY	Camera where object was found
	timestamp	TIMESTAMP	Time of detection
	obj_type	TEXT	Type of object (Bag, Box, Other)
	image_path	TEXT	Snapshot of detected object
vehicles	vehicle_id	SERIAL PRIMARY KEY	Unique ID for each detected vehicle
	camera_id	INTEGER FOREIGN KEY	Camera where vehicle was detected
	timestamp	TIMESTAMP	Time of detection
	plate_number	TEXT UNIQUE	Detected vehicle license plate
	vehicle_type	TEXT	Type of vehicle (Car, Truck, Motorcycle)
	image_path	TEXT	Snapshot of detected vehicle
video_storage	video_id	SERIAL PRIMARY KEY	Unique ID for video storage
	camera_id	INTEGER FOREIGN KEY	Camera that captured the video
	detected_at	TIMESTAMP	Time when video was recorded
	video_path	TEXT	File path of the stored video clip

	image_path	TEXT	File path of snapshot image
notifications	notif_id	SERIAL PRIMARY KEY	Unique notification ID
	user_id	INTEGER FOREIGN KEY	User receiving the notification
	alert_id	INTEGER FOREIGN KEY	Linked alert
	message	TEXT	Notification message
	sent_at	TIMESTAMP	Time notification was sent